COMPREHENSIVE ENGINEERING STUDY GUIDE & LAB MANUAL

Retrieval-Augmented Generation (RAG) & LangGraph Crash Course

Detailed Architecture Notes, Mathematical Foundations, Tool Calling, and Two Full Projects from Scratch (Managed with `uv`)

> This study manual covers the complete lifecycle of building grounded AI systems, starting from standard Retrieval-Augmented Generation (RAG) using free local embeddings (`sentence-transformers/all-MiniLM-L6-v2`) and `FAISS`, and advancing to stateful, multi-tool Agentic RAG workflows built with `LangGraph`. All code is production-ready for Python 3.11+ using `uv`.

Module 1: Foundations of Retrieval-Augmented Generation (RAG)

1.1 Why Pre-Trained LLMs Require RAG

Large Language Models store knowledge parametrically inside their neural weights during pre-training. While powerful at reasoning and language synthesis, standalone LLMs suffer from four foundational limitations in production environments:

- **Static Knowledge Cutoff:** A model cannot answer questions about events, policies, or research published after its training cutoff date.
- **Private Domain Blindness:** Pre-trained models have no access to proprietary company handbooks, student records, internal PDFs, or local SQL databases.
- **Extrinsic Hallucinations:** When prompted on unfamiliar topics, autoregressive models optimize for token plausibility rather than factual truth, fabricating citations and numbers.
- **High Cost of Retraining:** Fine-tuning weights just to inject new factual documents is computationally expensive and prone to catastrophic forgetting. *Engineering Rule: Use fine-tuning to teach formatting, tone, and classification schemas; use RAG to supply dynamic facts.*

1.2 The Two Core Pipelines of RAG

Every RAG architecture separates offline indexing from online inference so document processing never slows down user queries:

1. **Offline Ingestion & Indexing Pipeline:**
   - **Document Loading:** Parse raw files (PDF, Markdown, HTML, TXT) into structured `Document` objects containing `page_content` (string) and `metadata` (source filename, page number, section).
   - **Chunking (Text Splitting):** Break long documents into semantically cohesive passages using `RecursiveCharacterTextSplitter` so each chunk fits cleanly inside the embedding model's context window and represents a focused topic.

2. **Dense Vector Embedding:** Pass each text chunk through a local embedding model (`sentence-transformers/all-MiniLM-L6-v2`) to produce a 384-dimensional floating-point vector.
   - **Vector Store Persistence:** Index the vectors inside a local `FAISS` index and serialize it to disk (`vectorstore.save_local()`).
3. **Online Retrieval & Generation Pipeline:**
   - **Query Embedding:** Convert the user's natural language question into a 384-dimensional vector using the exact same embedding model used during ingestion.
   - **Similarity / MMR Search:** Compute vector similarity against the FAISS index to retrieve the Top-*K* most relevant document chunks.
   - **Prompt Augmentation:** Inject the retrieved chunks alongside source metadata and strict grounding instructions into the LLM system prompt.
   - **Grounded Generation:** The LLM synthesizes an answer strictly from the supplied context and cites the source metadata.

1.3 Chunking Hyperparameters & Trade-Offs

Chunking directly governs retrieval accuracy. If chunks are too small (e.g., 50 characters), sentences lose surrounding context and pronouns become ambiguous. If chunks are too large (e.g., 5,000 characters), multiple distinct topics blend into a single averaged embedding vector, reducing cosine similarity precision.

Parameter	Recommended Range	Engineering Purpose
`chunk_size`	400 – 800 characters	Keeps each chunk focused on a single paragraph or rule while staying well inside the 256-token window of compact embedding models.
`chunk_overlap`	60 – 150 characters (15%–20%)	Repeats boundary text across adjacent chunks so concepts split at the end of Chunk *i* remain intact at the start of Chunk *i+1*.
`separators`	`["\n\n", "\n", ". ", " ", ""]`	Splits hierarchically by paragraphs first, then newlines, then sentence boundaries before falling back to words.

1.4 Vector Embeddings & Similarity Mathematics

An embedding model maps discrete text into a continuous vector space $\mathbb{R}^d$ (where $d = 384$ for `all-MiniLM-L6-v2`). Given a query vector $\mathbf{q} = (q_1, q_2, \ldots, q_d)$ and a document chunk vector $\mathbf{d} = (d_1, d_2, \ldots, d_d)$, retrieval relies on two distance metrics:

$$\text{Cosine Similarity}(\mathbf{q}, \mathbf{d}) = (\mathbf{q} \cdot \mathbf{d}) \,/\, (\|\mathbf{q}\|_2 \, \|\mathbf{d}\|_2) = \sum_{i=1..d} (q_i d_i) \,/\, [\, \sqrt{(\sum q_i^2)} \times \sqrt{(\sum d_i^2)} \,]$$

$$\text{Squared Euclidean Distance } d_{L2}{}^2(\mathbf{q}, \mathbf{d}) = \|\mathbf{q} - \mathbf{d}\|_2{}^2 = \sum_{i=1..d} (q_i - d_i)^2$$

Why Normalization Matters: When you set `encode_kwargs={"normalize_embeddings": True}` in `HuggingFaceEmbeddings`, every vector is scaled to unit length ($\|\mathbf{q}\|_2 = 1$ and $\|\mathbf{d}\|_2 = 1$). Under unit normalization, Squared L2 Distance simplifies to $\|\mathbf{q} - \mathbf{d}\|_2{}^2 = 2 - 2(\mathbf{q} \cdot \mathbf{d})$, meaning FAISS L2 search produces the exact same ranking as Cosine Similarity.

1.5 Similarity Search vs. Maximal Marginal Relevance (MMR)

Standard Top-*K* similarity search often returns 3 nearly identical chunks from the same page. **Maximal Marginal Relevance (MMR)** solves this by fetching a larger candidate pool (`fetch_k=10`) and iteratively selecting `k=3` chunks that balance relevance to the query (λ) against diversity from already-selected chunks ($1 - \lambda$):

$$\text{MMR} = \arg\max_{D_i \in R \setminus S} [\, \lambda \cdot \text{Sim}(D_i, Q) - (1 - \lambda) \cdot \max_{D_j \in S} \text{Sim}(D_i, D_j) \,]$$

Module 2: Foundations of LangGraph & Agentic RAG

2.1 Why Linear RAG Chains Are Not Enough

A standard LangChain Expression Language (LCEL) pipeline executes a fixed sequence: `Query → Retriever → Prompt → LLM`. While ideal for single-purpose Q&A over static documents, linear chains have three major drawbacks:

- **Unnecessary Retrieval on Every Turn:** If a user says *"Hello, my name is Suman"* or *"Calculate 450 × 12"*, a fixed RAG chain still queries FAISS and stuffs irrelevant chunks into the prompt.
- **No Multi-Tool Orchestration:** Real-world queries often require combining internal policy documents (RAG) with live student database lookups, arithmetic calculation, and intent classification in a single request.
- **No Self-Correction Loop:** If the first search query returns zero useful chunks, a linear chain cannot reformulate the search query and try again.

2.2 Core Primitives of LangGraph

LangGraph models agent workflows as stateful, cyclic graphs where nodes read from and write updates to a typed shared state object:

LangGraph Concept	Technical Responsibility	Python Implementation
State (`AgentState`)	Shared schema passed across nodes. Extending `MessagesState` preserves message history plus custom counters like `llm_calls`.	`class AgentState(MessagesState): llm_calls: int`

LangGraph Concept	Technical Responsibility	Python Implementation
Reasoning Node (`llm_call`)	Invokes the tool-bound LLM with `SystemMessage` + `state["messages"]` and increments `llm_calls` .	`workflow.add_node("llm_call", llm_call)`
Action Node (`tool_node`)	Iterates through `state["messages"][-1].tool_calls` , invokes the matching Python function, and returns `ToolMessage` objects.	`workflow.add_node("tool_node", tool_node)`
Conditional Edge (`should_continue`)	Inspects the latest AI response. If `tool_calls` is non-empty, routes to `tool_node` ; otherwise terminates at `END` .	`workflow.add_conditional_edges("llm_call", should_continue, ["tool_node", END])`
Checkpointer (`MemorySaver`)	Persists graph state per `thread_id` so multi-turn follow-up queries retain full conversational memory.	`workflow.compile(checkpointer=MemorySaver())`

2.3 Anatomy of Reliable Custom Tools (`@tool`)

When you bind Python functions to an LLM via `model.bind_tools(tools)` , the LLM does not see your Python function body—it only sees the **function name**, **parameter type hints**, and **docstring** compiled into a JSON Schema. Follow three strict rules for every custom tool:

1. **Explicit Type Hints & `Literal` Constraints:** Use strict types (`amount: float` , `intent: Literal["policy", "math", "greeting"]` , `keywords: List[str]`) so the LLM formats arguments accurately.
2. **Action-Oriented Docstrings:** Clearly state *what* the tool returns and *when* the LLM should prefer it over other tools.
3. **Graceful Error Strings:** Catch exceptions inside the tool and return a helpful error string rather than crashing, allowing the LLM to self-correct its arguments on the next loop.

Module 3: Project Setup & Environment Management with `uv`

Both projects use **uv** for fast, reproducible dependency management and a 100% free local Hugging Face embedding model (`sentence-transformers/all-MiniLM-L6-v2`) paired with `FAISS` .

3.1 Initialize Project & Install Dependencies

```
# 1. Create a new directory and initialize uv
mkdir rag-langgraph-course && cd rag-langgraph-course
uv init

# 2. Install all required packages for Standard RAG, LangGraph, and Custom Tools
uv add \
  langgraph \
  langchain \
  langchain-core \
  langchain-community \
  langchain-text-splitters \
  langchain-google-genai \
  langchain-huggingface \
  langchain-tavily \
  sentence-transformers \
  faiss-cpu \
  wikipedia \
  ddgs \
  beautifulsoup4 \
  requests \
  pydantic \
  python-dotenv
```

3.2 Configure Environment Variables (`.env`)

Create a `.env` file in your project root directory:

```
GOOGLE_API_KEY="your_google_gemini_api_key_here"
TAVILY_API_KEY="your_optional_tavily_api_key_here"
```

Module 4: Project 1 — Complete Standard RAG Pipeline from Scratch

4.1 Project 1 Overview

Save the complete script below as `project1_standard_rag.py`. It implements the entire RAG lifecycle from scratch without needing external databases:

- Builds a realistic multi-page institutional & AI engineering knowledge base with metadata (`source` and `page`).
- Chunks documents using `RecursiveCharacterTextSplitter` (`chunk_size=450`, `chunk_overlap=80`).
- Embeds chunks locally using `HuggingFaceEmbeddings( model_name="sentence-transformers/all-MiniLM-L6-v2")` with unit L2 normalization and saves the index to disk (`faiss_standard_index/`).
- Executes MMR retrieval and generates grounded answers with mandatory source citations while refusing out-of-domain questions.

4.2 Complete Source Code: `project1_standard_rag.py`

```
"""
PROJECT 1: Complete Standard RAG Pipeline from Scratch
Run with: uv run --env-file .env python project1_standard_rag.py
"""

from pathlib import Path
from typing import List
from dotenv import load_dotenv

from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()

#  ======================================================================
# 1. CONFIGURATION
# ======================================================================
INDEX_DIR = "faiss_standard_index"
EMBEDDING_MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
CHUNK_SIZE = 450
CHUNK_OVERLAP = 80


# ======================================================================
# 2. INGESTION: SAMPLE KNOWLEDGE BASE DOCUMENTS
# ======================================================================
def load_sample_documents() -> List[Document]:
    """Creates structured domain documents with source and page metadata."""
    raw_pages = [
        {
            "source": "academic_policy_2026.pdf",
            "page": 1,
            "content": (
                "Attendance and Grading Policy (Grades 1 to 12): "
                "Students must maintain at least 80% attendance across all core
subjects "
                "(Mathematics, Science, English, Nepali, and Computer Science) to
sit for "
                "final examinations. A score of 90% or above earns Grade A+ (GPA
4.0), while "
                "80% to 89% earns Grade A (GPA 3.6). Any student scoring below 40%
in more "
                "than two subjects is enrolled in the 4-week Remedial Diagnostic
Lab."
            ),
        },
        {
```

Module 5: Project 2 — Complete Agentic RAG with LangGraph & Custom Tools

5.1 Project 2 Architecture & Execution Flow

Save the second script below as `project2_langgraph_rag.py`. It implements the exact LangGraph `llm_call` and `tool_node` pattern from our earlier sessions, combining a persistent FAISS RAG retriever with custom tools and conversational memory:

1. **Persistent FAISS Retriever Tool (`search_knowledge_base`):** Queries the local FAISS index for official handbook policies and technical specifications.
2. **7 Custom Domain & Utility Tools:**
   - `calculator`: Safe AST-based math evaluator for tuition waivers and averages.
   - `extract_intent_and_keywords`: Structured JSON classifier for user intent and key terms.
   - `greet_user`: Time-aware personalized greeting generator.
   - `lookup_student_profile`: Structured database lookup for student GPA, attendance, and family income.
   - `convert_currency`: Live exchange rate converter (e.g., USD to NPR).
   - `get_current_datetime`: System date, weekday, and time provider.
   - `save_advisor_report`: Safe local file writer for saving diagnostic reports.
3. **Stateful Graph Loop (`AgentState(MessagesState)`):** Tracks every message and counts total `llm_calls` across multi-step tool executions, backed by `MemorySaver` for multi-turn memory.

5.2 Complete Source Code: `project2_langgraph_rag.py`

```
"""
PROJECT 2: Complete Stateful Agentic RAG with LangGraph & Custom Tools
Run with: uv run --env-file .env python project2_langgraph_rag.py
"""

import ast
import json
import operator
import os
import requests
from datetime import datetime
from pathlib import Path
from typing import List, Literal
from dotenv import load_dotenv

from langchain_core.documents import Document
from langchain_core.messages import HumanMessage, SystemMessage, ToolMessage
from langchain_core.tools import tool
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_google_genai import ChatGoogleGenerativeAI
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.checkpoint.memory import MemorySaver

load_dotenv()

#  ======================================================================
# 1. INITIALIZE PERSISTENT LOCAL FAISS KNOWLEDGE BASE
# ======================================================================
INDEX_DIR = "faiss_langgraph_index"
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"

embeddings = HuggingFaceEmbeddings(
    model_name=EMBEDDING_MODEL,
    encode_kwargs={"normalize_embeddings": True},
)


def init_knowledge_base() -> FAISS:
    """Builds or loads the local FAISS index used by the RAG tool."""
    if Path(INDEX_DIR).exists():
        return FAISS.load_local(
            INDEX_DIR, embeddings, allow_dangerous_deserialization=True
        )

    docs = [
        Document(
            page_content=(
                "2026 Tuition & Scholarship Policy: Annual Grade 10 tuition is NPR
60,000. "
                "1) Merit Scholarship: 100% tuition waiver for students with GPA
>= 3.8 and "
```

Module 6: RAG Evaluation & Production Troubleshooting Reference

6.1 The RAG Evaluation Triad

Before deploying a RAG or LangGraph system to production, evaluate it across three orthogonal metrics:

- **Context Relevance (Query ↔ Retrieved Chunks):** Are the retrieved FAISS chunks directly focused on the user's question? Improve by tuning `chunk_size`, enabling `normalize_embeddings=True`, and switching to `search_type="mmr"`.
- **Faithfulness / Groundedness (Retrieved Chunks ↔ Answer):** Does every statement in the LLM's response come from the retrieved context? Enforce with `temperature=0` and explicit citation instructions.
- **Answer Relevance (Query ↔ Answer):** Did the agent complete all sub-tasks requested by the user (e.g., policy lookup + math calculation + intent extraction)?

6.2 Production Debugging & Architecture Cheat Sheet

Issue Observed	Root Cause	Engineering Fix
Slow tool execution on every RAG query (~3 seconds overhead)	`HuggingFaceEmbeddings` and `FAISS.load_local()` are called inside the `@tool` function body on every invocation.	Initialize `embeddings` and `VECTORSTORE` once at module startup (as shown in Project 2) or store them as fields on a `BaseTool` subclass.
`llm_calls` counter stays at `1` or disappears in LangGraph	Default `MessagesState` only tracks the `messages` key and drops undeclared dictionary keys.	Define `class AgentState(MessagesState): llm_calls: int` and pass `AgentState` to `StateGraph(AgentState)`.
Vector search misses exact course codes, IDs, or names	Dense embeddings compress semantic meaning and can miss rare alphanumeric strings.	Combine dense `FAISS` retrieval with keyword-based `BM25Retriever` using an ensemble hybrid retriever.
`ValueError` when loading local FAISS index from disk	FAISS uses Python pickle serialization for document metadata, which LangChain blocks by default for safety.	Pass `allow_dangerous_deserialization=True` inside `FAISS.load_local()` for trusted local indexes you generated yourself.